실험 전 보고서

LAB6_MCU2

실험 주제 : 컴퓨터와 FPGA 통신

실험 대상 : STM32-FPGA UART 통신 시스템

작성일 : 2026년 10월 25일

검증 환경 : STM32 개발보드, FPGA 개발보드, Serial Monitor(9600bps)

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. FPGA 실습 - 서보모터 제어

· **실험 목적**

본 실험의 목적은 FPGA를 이용하여 서보 모터를 정밀하게 제어하는 PWM(Pulse Width Modulation) 신호 생성 방법을 이해하고 구현하는 데 있다. 서보모터는 일반적으로 20 ms 주기의 PWM 신호를 사용하며, 펄스 폭에 따라 회전 각도가 결정된다. 따라서 FPGA 내부에서 정확한 시간 기준을 이용하여 1 ms, 1.5 ms, 2 ms 범위의 펄스 폭을 생성하고, 버튼 입력에 따라 펄스 폭을 변화시켜 서보모터의 위치를 제어하는 회로를 설계한다. 또한, 시뮬레이션을 통해 PWM 신호의 주기와 펄스 폭이 올바르게 생성되는지 검증하고, 위치 제한 기능 및 리셋 동작을 확인함으로써 FPGA 기반 모터 제어 시스템의 동작 원리를 학습한다.

· **핵심 코드 및 설명**

```
`timescale 1ns/1ps
module servo_ctrl #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output wire servo
);
    wire [31:0] pulse_us;
    servo_position u_position(clk_50mhz,rst_p,left_pulse,right_pulse,pulse_us);
    servo_pwm #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz,rst_p,pulse_us,servo);
endmodule
```

servo_ctrl 모듈은 서보모터 제어 시스템의 최상위 모듈로서, 위치 정보를 결정하는 servo_position 모듈과 PWM 신호를 생성하는 servo_pwm 모듈을 연결하는 역할을 수행한다. 입력으로 클록(clk_50mhz), 리셋(rst_p), 좌측 이동 펄스(left_pulse), 우측 이동 펄스(right_pulse)를 받아 현재 위치에 해당하는 펄스 폭 값(pulse_us)을 생성하고, 이를 PWM 발생기에 전달한다. 결과적으로 사용자의 버튼 입력에 따라 서보모터가 원하는 각도로 회전할 수 있도록 전체 제어 과정을 통합하는 역할을 담당한다.

```
`timescale 1ns/1ps

module servo_position(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output reg [31:0] pulse_us
);
    always @(posedge clk_50mhz)
        if(rst_p) pulse_us<=1500;
        else if(left_pulse && !right_pulse && pulse_us>1000) pulse_us<=pulse_us-100;
        else if(right_pulse && !left_pulse && pulse_us<2000) pulse_us<=pulse_us+100;
endmodule
```

servo_position 모듈은 버튼 입력에 따라 서보모터의 목표 위치를 결정하는 기능을 수행한다. 기본 위치는 1500μs(중앙 위치)로 설정되며, left_pulse가 입력되면 펄스 폭을 100μs 감소시키고 right_pulse가 입력되면 100μs 증가시킨다. 또한, 서보모터의 정상 동작 범위를 유지하기 위해 최소값인 1000μs와 최대값인 2000μs 사이로 동작 범위를 제한한다. 양쪽 버튼이 동시에 눌린 경우에는 위치를 변경하지 않도록 설계하여 오동작을 방지하였으며, 리셋 시에는 항상 중앙 위치로 복귀하도록 구현하였다.

```
module servo_pwm #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,
    input wire [31:0] pulse_us,
    output wire servo
);
    localparam FRAME_CYCLES=CLK_HZ/50; // 20 ms
    reg [31:0] count;
    reg [31:0] width_cycles;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; width_cycles<=(CLK_HZ/1000)*3/2; end
        else if(count==FRAME_CYCLES-1) begin
            count<=0;
            width_cycles<=pulse_us*(CLK_HZ/1000)/1000;
        end
        else count<=count+1;
    end
    assign servo=!rst_p && (count<width_cycles);
endmodule
```

servo_pwm 모듈은 servo_position에서 전달받은 펄스 폭 값을 이용하여 서보모터 구동용 PWM 신호를 생성한다. 서보모터의 표준 제어 방식에 맞추어 20ms 주기의 프레임을 생성하며, 프레임 내부에서 1ms~2ms 범위의 High 신호를 출력한다. 특히 새로운 위치 정보는 프레임 경계에서만 적용되도록 설계하여 PWM 출력 도중 펄스 폭이 변경되는 현상을 방지하였다. 이를 통해 서보모터가 안정적으로 동작하며, 펄스 폭에 따라 회

전 각도가 자연스럽게 변화하도록 구현하였다.

```
`timescale 1ns/1ps
module tb_servo;
    reg clk=0,rst=1,l=0,r=0;
    always #5 clk=~clk;
    wire [31:0] us;
    wire servo;
    servo_position position_dut(clk,rst,l,r,us);
    servo_pwm #(.CLK_HZ(100000)) pwm_dut(clk,rst,us,servo);
    integer i,high_count;
    task step; begin @(posedge clk); #1; end endtask
    task press(input bit left_button,input bit right_button);
        begin
            @(negedge clk); l=left_button; r=right_button; step;
            @(negedge clk); l=0; r=0;
        end
    endtask
    task check_frame(input integer expected_high);
        begin
            @(posedge servo); #1;
            high_count=0;
            for(i=0;i<2000;i=i+1) begin
                if(servo) high_count=high_count+1;
                step;
            end
            if(high_count != expected_high)
                $fatal(1,"servo width expected=%0d got=%0d",expected_high,high_count);
        end
    endtask
    initial begin
        step; if(us!==1500 || servo!==0) $fatal(1,"servo reset");
        @(negedge clk); rst=0;
        check_frame(150);
        repeat(8) press(left_button: 1,right_button: 0);
        if(us!==1000) $fatal(1,"left limit");
        check_frame(100);
        repeat(15) press(left_button: 0,right_button: 1);
        if(us!==2000) $fatal(1,"right limit");
        check_frame(200);
        press(left_button: 1,right_button: 1);
        if(us!==2000) $fatal(1,"both buttons");
        @(negedge clk); rst=1; step;
        if(us!==1500 || servo!==0) $fatal(1,"reset after running");
        $display("PASS servo: 20ms frame, 1/1.5/2ms pulse, limits, reset"); $finish;
    end
    initial begin #300000; $fatal(1,"timeout"); end
endmodule
```

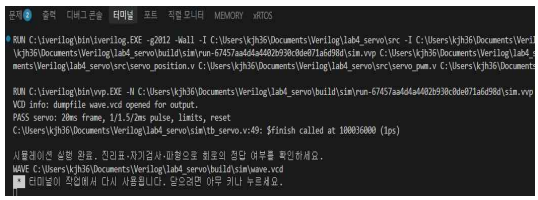
TB는 서보모터 제어 회로가 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 리셋 상태에서 초기 펄스 폭이 1500μs로 설정되는지 확인하고, 좌우 버튼 입력에 따라 1000μs와 2000μs의 한계값까지 정상적으로 증가 및 감소하는지를 검사한다. 또한, 20ms PWM 프레임 내에서 1ms, 1.5ms, 2ms에 해당하는 High 구간의 클록 수를 측정하여 PWM 신호가 정확하게 생성되는지 검증한다. 마지막으로 양쪽 버튼 동시 입력과 리셋 동작까지 확인하여 전체 서보모터 제어 시스템의 신뢰성을 검증하도록 구성하였다.

· **실행 결과**

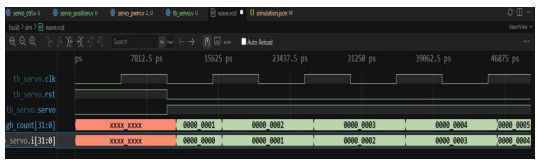
- Tool



- Simulation



- Wave



2. FPGA 실습 - SRAM 읽기/쓰기

· **실험 목적**

본 실험의 목적은 FPGA와 SRAM(Static Random Access Memory) 간의 인터페이스를 구현하여 데이터의 저장(Write)과 읽기(Read) 동작을 이해하는 데 있다. SRAM의 주소선(Address Bus), 데이터선(Data Bus), 제어신호(WE, OE, CE)의 동작 원리를 학습하고, FPGA를 이용하여 특정 주소에 데이터를 기록한 후 다시 읽어 정상적으로 저장되었는지 확인한다. 또한, 읽기와 쓰기 과정에서 발생하는 타이밍 특성과 버스 제어 방법을 이해하고, 시뮬레이션 및 실제 동작 검증을 통해 메모리 제어 회로 설계 능력을 향상

시키는 것을 목표로 한다.

· 핵심 코드 및 설명

```
`timescale 1ns/1ps

module sram(
    input wire ctrl_w,ctrl_r,
    input wire [3:0] address,data_in,
    output wire ram_cs,ram_oe,ram_we,
    output wire [17:0] ram_address,
    inout wire [15:0] ram_data,
    output wire [7:0] led_out
);
    wire drive_data,read_data;
    sram_bus_control u_control(ctrl_w,ctrl_r,drive_data,read_data,ram_cs,ram_oe,ram_we);
    assign ram_address={14'b0,address};
    assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
    assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
endmodule
```

SRAM 모듈은 메모리의 읽기(Read) 및 쓰기(Write) 동작을 구현한 핵심 회로이다. 내부에 메모리 배열을 가지고 있으며, 입력된 주소(Address)에 따라 데이터를 저장하거나 출력한다. 쓰기 신호가 활성화되면 데이터 버스를 통해 입력된 값을 해당 주소에 저장하고, 읽기 신호가 활성화되면 지정된 주소에 저장된 데이터를 출력한다. 이를 통해 실제 SRAM 장치의 동작을 FPGA 내부에서 모사하며, 주소 지정에 따른 데이터 접근 방법과 메모리 동작 원리를 학습할 수 있도록 설계되었다.

```
`timescale 1ns/1ps

module sram_bus_control(
    input wire ctrl_w,ctrl_r,
    output wire drive_data,read_data,
    output wire ram_cs,ram_oe,ram_we
);
    assign drive_data=ctrl_w && !ctrl_r;
    assign read_data=ctrl_r && !ctrl_w;
    assign ram_cs=!(drive_data || read_data);
    assign ram_oe=!read_data;
    assign ram_we=!drive_data;
endmodule
```

sram_bus_control 모듈은 SRAM과 FPGA 사이의 양방향 데이터 버스를 제어하는 역할을 수행한다. SRAM의 데이터선은 읽기와 쓰기에 모두 사용되므로, 상황에 따라 FPGA가 데이터를 출력하거나 입력받을 수 있어야 한다. 쓰기 동작 시에는 FPGA가 데이터 버스를 구동하여 SRAM에 데이터를 전달하고, 읽기 동작 시에는 FPGA의 출력단을 High-Z(고임피던스) 상태로 전환하여 SRAM이 데이터를 출력할 수 있도록 한다. 이러한 Tri-State Buffer 구조를 통해 하나의 데이터 버스를 공유하면서도 버스 충돌을 방지할 수 있으며, 안정적인 메모리 통신을 구현할 수 있다.

```
`timescale 1ns/1ps

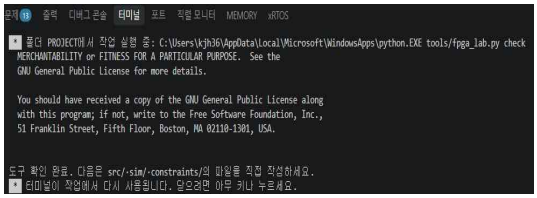
module sram_model(
    input wire cs_n,oe_n,we_n,
    input wire [17:0] address,
    inout wire [15:0] data
);
    reg [15:0] memory[0:15];
    assign data=(!cs_n && !oe_n && we_n) ? memory[address[3:0]] : 16'bz;
    always @(negedge we_n) begin
        #1;
        if(!cs_n && !we_n) memory[address[3:0]]=data;
    end
endmodule

module tb_sram;
    reg wr=0,rd=0;
    reg [3:0] address=0,data_in=0;
    wire cs_n,oe_n,we_n;
    wire [17:0] ram_address;
    wire [15:0] bus;
    wire [7:0] led;
    sram dut(wr,rd,address,data_in,cs_n,oe_n,we_n,ram_address,bus,led);
    sram_model memory(cs_n,oe_n,we_n,ram_address,bus);
    integer i;
    initial begin
        #2;
        if(bus !== 16'bz || led !== 0) $fatal(1,"idle");
        for(i=0;i<16;i=i+1) begin
            address=i; data_in=15-i; #2;
            wr=1; #2;
            if(bus !== 15-i || ram_address !== i) $fatal(1,"write bus");
            wr=0; #2;
        end
        for(i=0;i<16;i=i+1) begin
            address=i; rd=1; #2;
            if(led !== 15-i || bus !== 15-i) $fatal(1,"read address=%0d",i);
            rd=0; #2;
        end
        wr=1; rd=1; #2;
        if(bus !== 16'bz || {cs_n,oe_n,we_n} !== 3'b111) $fatal(1,"conflict");
        $display("PASS SRAM: 16 addresses, bus release, simultaneous request"); $finish;
    end
    initial begin #1000; $fatal(1,"timeout"); end
endmodule
```

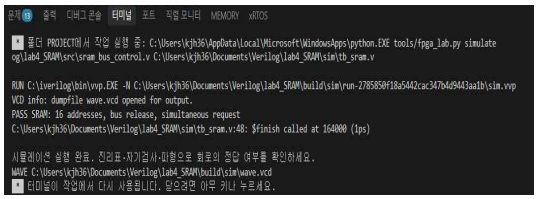
TB는 SRAM 및 sram_bus_control 모듈이 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 특정 주소에 데이터를 기록한 후 동일한 주소에서 데이터를 읽어 저장된 값과 일치하는지 확인하며, 여러 주소에 대해 반복적으로 읽기·쓰기 동작을 수행하여 메모리 제어 기능을 검증한다. 또한, 데이터 버스가 읽기 모드에서는 High-Z 상태로 전환되고, 쓰기 모드에서는 정상적으로 데이터를 출력하는지 확인함으로써 버스 제어 로직의 정확성을 검사한다. 이를 통해 실제 FPGA 구현 전에 설계 오류를 발견하고 SRAM 인터페이스의 신뢰성을 검증할 수 있다.

· **실행 결과**

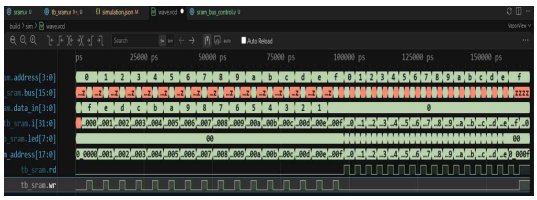
- Tool



- Simulation



- Wave



3. FPGA 실습 - DAC

· **실험 목적**

본 실험의 목적은 DAC(Digital to Analog Converter)의 동작 원리를 이해하고 FPGA를 이용하여 디지털 데이터를 아날로그 신호로 변환하는 방법을 학습하는 데 있다. FPGA에서 생성한 디지털 값을 DAC에 입력하여 이에 대응하는 아날로그 전압을 출력하고, 입력 데이터 변화에 따른 출력 전압의 특성을 확인한다. 또한, DAC의 해상도(Resolution), 기준 전압(Reference Voltage), 변환 정확도와 같은 주요 개념을 이해하고, 시뮬레이션 및 실제 파형 측정을 통해 디지털 시스템과 아날로그 시스템 간의 인터페이스 동작을 검증한다. 이를 통해 FPGA 기반 신호 생성 및 데이터 변환 기술을 습득하고, 향후 오디오 출력, 센서 제어, 파형 발생기 등의 응용 시스템 설계에 필요한 기초 지식을 확보하는 것을 목표로 한다.

· **핵심 코드 및 설명**

```
`timescale 1ns/1ps

module dac_parallel8(
    input wire clk_50mhz,rst_p,update,channel,
    input wire [7:0] value,
    output wire cs_n,ldac_n,
    output reg we_n,a_b,
    output reg [7:0] data
);
    assign cs_n=0;
    assign ldac_n=0;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin we_n<=1; a_b<=0; data<=0; end
        else begin
            we_n<=1;
            if(update) begin data<=value; a_b<=channel; we_n<=0; end
        end
    end
endmodule
```

dac_parallel8 모듈은 8비트 디지털 데이터를 DAC에 전달하기 위한 병렬 인터페이스 회로이다. FPGA 내부에서 생성된 8비트 데이터 값을 DAC 입력 핀에 직접 연결하여 출력하며, 각 비트의 상태에 따라 DAC가 대응되는 아날로그 전압을 생성할 수 있도록 한다. 즉, 디지털 신호와 DAC 하드웨어 사이의 데이터 전달 경로를 구성하는 역할을 수행하며, 입력 데이터가 변경될 때마다 DAC 출력 전압도 함께 변화하도록 설계되었다.

```
1  `timescale 1ns/1ps
2  module dac #(parameter UPDATE_CYCLES=500_000)(
3      input wire clk_50mhz,rst_p,add_sel,
4      output wire dac_csn,dac_ldacn,dac_wen,dac_a_b,
5      output wire [7:0] dac_d,led_out
6  );
7      wire [7:0] value;
8      wire update;
9      ramp8 #(.UPDATE_CYCLES(UPDATE_CYCLES)) u_ramp(clk_50mhz,rst_p,value,update);
10     dac_parallel8 u_bus(clk_50mhz,rst_p,update,add_sel,value,
11         dac_csn,dac_ldacn,dac_wen,dac_a_b,dac_d);
12     assign led_out=dac_d;
13 endmodule
```

dac 모듈은 DAC 시스템의 최상위 제어 회로로서 디지털 데이터 생성부와 DAC 인터페이스를 연결하는 역할을 수행한다. 입력 클록을 기준으로 생성된 디지털 값을 DAC

장치로 전달하며, 필요한 제어 신호와 데이터 흐름을 관리한다. 또한 FPGA 내부 로직과 외부 DAC 회로 간의 인터페이스를 담당하여 디지털 값이 정확한 아날로그 전압으로 변환될 수 있도록 전체 동작을 제어한다.

```
`timescale 1ns/1ps
module ramp8 #(parameter UPDATE_CYCLES=500_000)(
    input wire clk_50mhz,rst_p,
    output reg [7:0] value,
    output wire update
);
    reg [31:0] count;
    assign update=(count==UPDATE_CYCLES-1);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; value<=0; end
        else if(update) begin count<=0; value<=value+1'b1; end
        else count<=count+1;
    end
endmodule
```

ramp8 모듈은 8비트 카운터를 이용하여 램프(Ramp) 형태의 데이터를 생성하는 회로이다. 클록 신호에 따라 출력값이 0부터 255까지 순차적으로 증가하며, 최대값에 도달하면 다시 0부터 반복적으로 증가한다. 이 데이터를 DAC 입력으로 사용하면 출력 전압이 계단 형태로 점차 증가하는 램프 파형이 생성된다. 이를 통해 DAC가 디지털 값의 변화에 따라 아날로그 전압을 정상적으로 출력하는지 확인할 수 있으며, DAC 동작 검증용 신호 발생기의 역할을 수행한다.

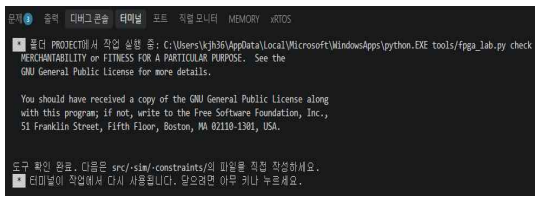
```
`timescale 1ns/1ps
module tb_dac;
    reg clk=0,rst=1,channel=0;
    always #5 clk=~clk;
    wire cs_n,ldac_n,we_n,a_b;
    wire [7:0] data,led;
    dac #(.UPDATE_CYCLES(4)) dut(clk,rst,channel,cs_n,ldac_n,we_n,a_b,data,led);
    integer writes=0;
    time last_fall;
    reg [7:0] held_data;
    reg held_channel;

    always @(negedge we_n) begin
        #1; held_data=data; held_channel=a_b; last_fall=$time;
    end
    always @(posedge we_n) begin
        if(!rst) begin
            if($time-last_fall != 9) $fatal(1,"DAC write pulse width");
            if(data !== held_data || a_b !== held_channel) $fatal(1,"DAC data hold");
            if(data !== (writes%256) || led !== data) $fatal(1,"DAC ramp %0d",writes);
            if(cs_n!==0 || ldac_n!==0 || a_b!==channel) $fatal(1,"DAC controls");
            writes=writes+1;
        end
    end
    initial begin
        repeat(2) @(negedge clk); rst=0;
        wait(writes==130); @(negedge clk); channel=1;
        wait(writes==260); @(negedge clk); rst=1;
        @(posedge clk); #1;
        if(we_n!==1 || data!==0) $fatal(1,"DAC reset");
        $display("PASS DAC: 260 writes, wraparound, channel, data hold, reset"); $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

TB는 dac_parallel8, dac, ramp8 모듈이 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 클록 신호를 생성하여 ramp8 모듈의 카운트 값이 순차적으로 증가하는지 확인하고, 생성된 데이터가 DAC 인터페이스를 통해 올바르게 전달되는지를 검증한다. 또한 디지털 출력값의 변화가 예상된 순서대로 발생하는지 관찰하여 DAC 입력 데이터 생성 과정의 정확성을 검사한다. 이를 통해 실제 FPGA 구현 전에 기능적 오류를 발견하고 DAC 시스템의 신뢰성을 검증할 수 있다.

· 실행 결과

- Tools



- Simulation



- Wave



4. STM32_2 - 과제 5

· 실험 목적

본 실험의 목적은 UART(Universal Asynchronous Receiver Transmitter) 통신을 이용하여 PC와 STM32 마이크로컨트

롤러 간의 데이터 송수신 방법을 이해하고, 시리얼 통신을 통해 입력된 문자를 정확하게 수신 및 처리하는 과정을 학습하는 데 있다. 또한 PC에서 입력한 문자 데이터를 STM32가 수신한 후 해당 문자의 ASCII 코드 값을 출력함으로써 문자 데이터와 이진 데이터의 관계를 확인하고, UART 기반 통신 시스템의 동작 원리를 검증하는 것을 목표로 한다. 이를 통해 이후 MCU-FPGA 간 UART 통신 실습을 위한 기초적인 데이터 송수신 기법을 습득한다.

· 핵심 코드 및 설명

```c
/* 메인 루프에서 반복해서 호출됩니다. */
void polling_routine(void)
{
    int ch = getchar();

    if(ch != EOF)
    {
        printf("문자 : %c, HEX : 0x%02X\r\n",
               ch,
               (uint8_t)ch);
    }
}
```

UART를 통해 PC로부터 입력된 문자를 수신하고, 수신된 데이터가 존재할 경우 문자 형태와 ASCII 코드(16진수)를 함께 출력한다. getchar() 함수를 이용하여 문자 데이터를 읽고, %c를 통해 문자 형태로 출력하며 %02X를 이용하여 ASCII 코드를 16진수 형식으로 표시한다. 이를 통해 UART 통신 데이터와 ASCII 코드의 대응 관계를 확인할 수 있다.

· 예상 결과

PC의 시리얼 모니터를 통해 입력한 문자가 STM32에 의해 정상적으로 수신되고, 수신된 문자와 해당 ASCII 코드가 16진수 형태로 출력되는 결과를 확인할 수 있다. 예를 들어 사용자가 문자 A를 입력하면 문자 : A, HEX : 0x41, 숫자 문자 1을 입력하면 문자 : 1, HEX : 0x31, 소문자 a를 입력하면 문자 : a, HEX : 0x61이 출력된다. 이를 통해 UART 통신이 정상적으로 동작함을 확인할 수 있으며, 문자 데이터가 컴퓨터 내부에서 ASCII 코드 형태로 처리된다는 사실을 검증할 수 있다. 또한, 다양한 문자를 입력하여 송수신 결과를 비교함으로써 STM32의 UART 수신 기능과 데이터 처리 과정이 정확하게 구현되었음을 확인할 수 있다.

5. STM32_2 - 과제 6

· 실험 목적

UART 통신을 이용하여 STM32와 FPGA 간의 데이터 전송 기능을 구현하고 검증하는 것을 목적으로 한다. PC에서 입력한 문자 데이터를 STM32가 수신한 후 UART3를 통해 FPGA로 전달하고, FPGA가 수신한 데이터를 다시 출력함으로써 MCU와 FPGA 간의 UART 통신이 정상적으로 동작하는지 확인한다. 이를 통해 이종 디지털 시스템 간 데이터 인터페이스의 동작 원리를 이해하고 UART 기반 통신 기술을 학습한다.

· 핵심 코드 및 설명

```c
void polling_routine(void)
{
    int ch = getchar();

    if(ch != EOF)
    {
        uint8_t data = (uint8_t)ch;

        HAL_StatusTypeDef result =
            HAL_UART_Transmit(
                &huart3,
                &data,
                1U,
                100U
            );

        printf("TX Result : %d\r\n", result);
    }
}
```

PC로부터 UART를 통해 입력된 문자를 수신한 후 HAL_UART_Transmit() 함수를 이용하여 UART3 인터페이스를 통해 FPGA로

전송한다. 입력된 데이터는 1바이트 크기로 변환되어 FPGA에 전달되며, 전송 결과는 반환값을 통해 확인한다. 이를 통해 STM32가 UART 브리지 역할을 수행하며 PC와 FPGA 사이의 데이터 전달이 정상적으로 이루어지는지 검증할 수 있다.

· 예상 결과

PC의 시리얼 모니터를 통해 입력한 문자 데이터가 STM32에 의해 정상적으로 수신된 후 UART3를 통해 FPGA로 전송되고, FPGA가 해당 데이터를 수신하여 다시 PC로 출력하는 결과를 확인할 수 있다. 예를 들어 사용자가 A, 1, B 등의 문자를 입력하면 FPGA 측 시리얼 모니터에 동일한 문자가 출력되며, STM32에서는 전송 성공 시 TX Result : 0 메시지가 표시된다. 이를 통해 STM32와 FPGA 간 UART 통신 연결이 정상적으로 이루어지고 있음을 확인할 수 있으며, PC → STM32 → FPGA로 이어지는 데이터 전송 경로가 정확하게 동작함을 검증할 수 있다. 또한, 다양한 문자 입력에 대해 동일한 결과가 출력됨으로써 데이터 손실이나 오류 없이 송수신이 수행됨을 확인할 수 있다.

6. STM32_2 - 과제 7

· 실험 목적

UART 통신을 통해 FPGA가 수신한 데이터를 이용하여 LED를 제어하는 기능을 구현하고 검증하는 것을 목적으로 한다. PC에서 입력한 명령 문자를 STM32와 FPGA를 거쳐 전달하고, FPGA가 수신한 데이터에 따라 LED의 ON, OFF 및 TOGGLE 동작을 수행함으로써 UART 기반 제어 시스템의 동작 원리를 이해하고 디지털 입출력 제어 능력을 학습한다.

· 핵심 코드 및 설명

```
module top(
    input clk,
    input rx,
    output tx,
    output led
);
```

Top Module(최상위 모듈)을 수정한다.

```
reg led_reg;
```

```
assign led = led_reg;
```

LED 상태 저장용 레지스터 추가 및 출력을 연결한다.

```
wire [7:0] rx_data;
wire rx_done;
```

UART 수신 데이터 연결을 확인한다.

```
always @(posedge clk)
begin
    if (rx_done)
    begin
        case (rx_data)

            8'h30:
                led_reg <= 1'b0;

            8'h31:
                led_reg <= 1'b1;

            8'h32:
                led_reg <= ~led_reg;

            default:
                led_reg <= led_reg;

        endcase
    end
end
```

UART를 통해 수신된 문자 데이터를 확인하여 LED의 상태를 제어한다. 수신 완료 신

호가 발생하면 입력된 데이터를 검사하고, 문자 '0'이 입력되면 LED를 OFF, 문자 '1'이 입력되면 ON, 문자 '2'가 입력되면 현재 LED 상태를 반전(Toggle)하도록 동작한다.

· **예상 결과**

PC에서 시리얼 통신을 통해 입력한 명령 문자가 STM32를 거쳐 FPGA로 전달되고, FPGA가 수신한 데이터에 따라 LED를 제어하는 결과를 확인할 수 있다. 입력 문자 '1'을 전송하면 LED가 점등되고, '0'을 전송하면 LED가 소등되며, '2'를 전송하면 현재 LED 상태가 반전되어 ON과 OFF가 반복적으로 전환된다. 또한 정의되지 않은 다른 문자를 입력할 경우 LED 상태는 변화 없이 유지된다. 이를 통해 UART 통신으로 수신한 데이터를 FPGA가 정상적으로 해석하여 외부 출력장치를 제어할 수 있음을 확인할 수 있으며, PC-STM32-FPGA로 구성된 UART 기반 제어 시스템의 동작을 검증할 수 있다.

7. 출처

· STM icroelectronics, STM32 Reference Manual (RM0440), STMicroelectronics, 2024.

· Douglas V. Hall, Microprocessors and Interfacing: Programming and Hardware, McGraw-Hill Education, 2nd Edition.

8. Git-Hub 및 최종 커밋

https://github.com/UnnamedDays/ECE-II-Project

b958ba8